```
$ docker-compose up -d webpack_dev_server
Recreating myapp_webpack_dev_server_1 ... done
```

Since we're using the same gem_cache volume for both our web and webpack_dev_server services, gems added by updating the web service will automatically be available to the webpack_dev_server service, and vice versa.

Let's review the benefits and downsides to this strategy:

Pros:

- Speeds up gem management for all Bundler actions: adding, removing, or updating gems
- Uses familiar bundle install workflow predominantly during development

Cons:

- Bundle commands only update our local volume; we still ultimately need to build the image
- Possibility for confusion over which gems are being loaded or used
- Extra complexity of changes to both Dockerfile and docker-compose.yml, plus the need to understand the nuance of gems being overlaid in local volume

Many Rails developers will find this approach appealing because we explicitly manage our gems with Bundler like we're used to, instead of relying on rebuilding our image to run bundle install. If you can get your head around the complexity, this strategy will definitely make your local development feel snappier, especially if you're making a lot of gem changes.

Quick Recap

Who knew that gem management could be such a hot topic? With this new option at your disposal, you should feel ready for anything.

Let's take a look back at what we covered in this chapter:

1. We discussed the downsides to our previous approach to managing gems: any gem changes required all gems to be reinstalled from scratch.
2. We explored a way to speed up gem changes that involved creating a volume for caching our gems. By mounting the volume into our container (and setting BUNDLE_PATH), we could manage our gems manually and benefit from faster builds.

Hopefully, you're also starting to appreciate the meta point here: how problems with our Docker setup can be identified, thought through, and eventually